LLM Application Development with LangChain and LangGraph

A theory-focused reference covering the core concepts involved in building practical LLM applications: the LangChain component model, graph-based workflow orchestration with LangGraph, retrieval-augmented generation, tool usage and agent loops, conversational memory and persistence, and advanced execution patterns such as parallel workflows, subgraphs and fault tolerance.

---

1. Foundations of LLM Applications

1.1 LangChain and its ecosystem

LangChain is a framework for composing applications around large language models. It provides a set of standardised building blocks — models, prompts, parsers, tools, and agents — that can be connected in different arrangements without rewriting glue code for each model provider.

The broader ecosystem includes:

- **LangChain core** – the foundational abstractions: chat models, prompt templates, output parsers, runnables, messages, documents and tools.
- **Partner integrations** – provider-specific packages (for example, packages for Ollama or Google Gemini) that implement the same core interfaces, so switching providers does not change application code.
- **LangGraph** – a library for building stateful, resumable workflows as graphs (nodes and edges) on top of LangChain components. Where LangChain supplies the *parts*, LangGraph supplies the *control flow* over those parts.

1.2 General architecture of a model-powered pipeline

The simplest LLM application follows a linear pipeline:

```
input → prompt template → chat model → output parser → output
```

1. **Input** – the user's raw text (a question, a request, a record to analyse).
2. **Prompt template** – formats the input into a structured message list, usually a system message (instructions, persona, rules) plus a human message (the actual question).
3. **Chat model** – sends the messages to the model (local or cloud) and returns a message object.
4. **Output parser** – converts the model's message content into a plain string or a structured object the rest of the program can use.
5. **Output** – displayed to the user or passed to the next stage.

More advanced applications extend this pipeline with retrieval steps (RAG), tool execution loops (agents), routing (classification and conditional execution) and persistence (memory across turns).

1.3 Chat models

A chat model is an object that exposes a standard interface for talking to an LLM:

- `.invoke(messages)` – send a list of messages, get a single response back.
- `.stream(messages)` – yield response chunks as they are generated.
- `.batch([...])` – run several requests together.
- `.with_structured_output(schema)` – request a response that matches a given schema and return a validated object.

Responses are **message objects** (for example `HumanMessage`, `AIMessage`, `SystemMessage`, `ToolMessage`), not raw strings. Each message has a role and content. Content may be a plain string or a list of content parts, depending on the provider — parsers handle both forms.

Two common deployment modes:

Mode	Example	Characteristics
Local	Ollama server on localhost	No API key, no per-call cost, limited by local RAM/CPU, requires the model to be "pulled" first
Cloud	Gemini API	Requires an API key, metered quota (often with a free tier), stronger models, network dependency

Both modes are accessed through the same LangChain chat-model interface, so application code does not change when the provider changes.

Temperature is a sampling parameter: low values (e.g. 0.2) make answers more deterministic and consistent — desirable for finance, policy or compliance answers — while higher values produce more varied wording.

1.4 Prompt templates

A prompt template is a reusable message structure with placeholders:

```
system: <instructions / persona / rules>
human:  {question}
```

Key ideas:

- **Roles.** System messages define behaviour (who the assistant is, what it may and may not do, formatting rules). Human messages carry the user's request.

- **Placeholders.** Text such as `{question}` or `{context}` is filled in at invocation time via `.invoke({"question": "..."})` or `.format(...)`.
- **Separation of content and logic.** Prompt wording is business content. Keeping all system prompts in one module makes answers consistent across different versions of an application, allows non-programmers to edit wording, and lets different workflow branches use different prompts (for example, one prompt per query category).
- **Composable identity blocks.** A shared identity/behaviour block can be prepended to every category-specific prompt so rules stay uniform while the task focus varies.

1.5 Output parsers

The chat model returns a message object; most applications need a plain string (or a structured object). Output parsers perform that conversion:

- **`StrOutputParser`** – extracts `.content` from an `AIMessage` and returns a string. It safely handles both string content and list-of-parts content.
- **Structured parsers** – convert model output into Pydantic models or JSON objects (often combined with schema-guided generation, see below).

Parsers are the final stage of a composed chain, so downstream code always receives a predictable type.

1.6 Structured outputs

Free text cannot be validated, stored in a typed field, or used for routing. Structured output asks the model to return data that matches an explicit schema.

Mechanism:

1. Define a schema — typically a Pydantic `BaseModel` with typed fields and field descriptions. Field descriptions are part of the prompt the model sees; they explain what to put in each field. `Literal[...]` restricts a field to a fixed set of allowed values.
2. Call `llm.with_structured_output(Model)`. LangChain converts the schema into the provider's native mechanism:
   - **function/tool calling** (common on Gemini and newer models),
   - **JSON-schema mode** (common on local models via Ollama).
3. The reply is validated into a Pydantic instance. If the model returns something invalid, validation raises an error.

Why both LLM and code are validated against the same list: using a `Literal` for categories means the model's output *and* the Python routing code share one vocabulary — an unexpected category string fails validation instead of silently breaking the graph.

Fallback pattern. Because structured output support varies across models (especially smaller local models), robust applications wrap the structured call in `try/except` and fall back to either plain text or a

deterministic pure-Python alternative (for example, a keyword-based classifier). The application then *always* gets a usable result.

Typical uses of structured output:

- classification (assign a category + reasoning to a query),
- planning (a list of typed tasks),
- formatted answers (category + answer + key points).

1.7 Runnables and LCEL (LangChain Expression Language)

LCEL is a declarative syntax for composing components into a single **Runnable** object using the pipe operator `|`:

```
chain = prompt | llm | StrOutputParser()
```

Properties of a Runnable:

- **One interface** – `.invoke()`, `.stream()`, `.batch()` work the same on every component and on every composed sequence.
- **Sequential execution** – `chain.invoke({...})` runs prompt → model → parser in order and returns the final result.
- **Composability** – chains can be chained; structured models (`llm.with_structured_output(...)`) are themselves runnables and slot into the same syntax.
- **Separation from orchestration** – a chain is a linear pipeline; LangGraph decides *when* and *in what order* such pipelines (or other functions) run.

Building a chain is cheap (it only wires objects together); only `.invoke()` costs a model call. Applications therefore build the chain once and reuse it for every user turn.

1.8 Configuration management best practices

LLM applications need externalised configuration:

- **`.env` file** – the single place where provider selection, model names, API keys, embedding model names and persistence settings live. It is local configuration and should never be committed or pasted into logs; a shareable `.env.example` with placeholders serves as the template.
- **Configuration module** – a dedicated Python module locates the `.env` file, loads it (shell environment variables typically take precedence, so quick overrides still work), and publishes values as module-level constants. All other files import from this module; nothing hardcodes keys or model names.
- **Validation before use** – the config module checks that the selected provider actually has its required settings (model name, API key) and fails fast with an actionable message *before* any network call. Placeholder values are detected and rejected.

- **Factory functions** – a model factory (`get_llm()` , `get_embeddings()`) constructs the provider-specific object from validated config. Provider integration packages are imported *lazily* inside the matching branch, so a missing optional dependency only breaks the provider that needs it, and both providers return objects with the identical interface.
- **Hints on failure** – runtime connection errors surface actionable hints built from the *current* configuration (is the local server running? is the model pulled? is the key set?), never from hardcoded strings.

Result: switching between a local and a cloud provider is a one-line config change with zero workflow code changes.

1.9 Tools and agents (overview)

- **Tools** are ordinary Python functions (calculators, database lookups, API calls) that a model may request. They carry a schema the model reads to decide *when* and *with what arguments* to call them. Python — not the model — executes the function and returns the exact result.
- **Agents** are loops of: *model decides → tool runs → result fed back to model → model decides again*, repeating until the model produces a final answer. This pattern is what lets an LLM perform multi-step work it cannot do reliably in a single shot (especially arithmetic and record lookups).

These ideas are covered in full in Sections 5 and 7.

---

2. LangGraph Fundamentals

2.1 Why graphs instead of straight chains

A linear chain (prompt → model → parser) is enough for a single-shot question. Real applications need:

- **Branching** – different paths depending on the data (route a classified invoice question to an invoice handler).
- **Loops** – repeat until a condition is met (call a tool, feed the result back, ask again).
- **State** – data that persists across steps and is shared by all steps.
- **Human input** – pause for approval, then resume.
- **Parallelism** – run independent tasks concurrently.
- **Resumability** – survive restarts and continue mid-workflow.

A single growing function expresses none of this cleanly. LangGraph models the application as a **graph of nodes connected by edges**, with an explicit **state** that flows through it.

2.2 Core concepts

Concept	Definition
State	A shared data structure (typically a `TypedDict`) that is the single source of truth every node reads and may update

Concept	Definition
Node	A plain Python function `(state) -> dict-of-updates`. May call a chain, run pure Python, or execute tools
Edge	A connection determining what runs next. May be unconditional (A always → B) or conditional (decide at runtime)
START / END	Special nodes marking entry and exit of the graph
StateGraph	The builder class: register nodes and edges, then `compile()`
Compiled graph	The runnable application returned by `compile()`; supports `.invoke()` and `.stream()`

2.3 State in detail

- Declared as a `TypedDict` — a static description of what exists in the workflow (keys and types).
- Nodes receive the **current merged state** and return a dictionary containing **only the keys they want to change**. LangGraph merges that partial update into the state; untouched keys keep their previous values.
- Returning updates (rather than mutating the state in place) is the contract between the framework and node code.
- Declaring fields also serves as validation: a node returning an undeclared key raises an error, catching typos early.
- **Default merge semantics are overwrite (last-write-wins)** — correct when each key has a single writer per run (e.g. `category`, `final_output`). When multiple writers touch the same key, a **reducer** is required (Section 3.2).

2.4 Building and running a graph

Typical construction:

```
builder = StateGraph(FinanceState)
builder.add_node("step_a", fn_a)
builder.add_node("step_b", fn_b)
builder.add_edge(START, "step_a")
builder.add_edge("step_a", "step_b")
builder.add_edge("step_b", END)
app = builder.compile()
```

- **`compile()`** validates the graph (reachability, missing paths) and returns the runnable. Forgetting it (or forgetting `invoke()`) is a common source of "the graph does nothing" bugs.
- **`app.invoke(initial_state)`** runs START → END once and returns the full final state.

- `app.stream(state, stream_mode="updates")` yields one dictionary per executed node: `{node_name: {keys it changed}}`. This is invaluable for *visualising execution* — you see the order of nodes and exactly what each contributed. Merging those partial updates into a local copy mirrors what LangGraph does internally, so the final answer needs only one model call.

2.5 Sequential workflow pattern

A minimal but complete graph pattern:

```
START → input processing → model response → output processing → END
```

- **Input processing** – pure Python: normalise text, seed the state (e.g. push the user's message into a message list). No model call.
- **Model response** – reads the cleaned input, runs the composed chain, stores the raw answer.
- **Output processing** – presentation only: format the final reply so any UI can rely on one field.

Even when functionally identical to a straight chain, structuring the work as a graph pays off immediately: every later feature (routing, memory, parallelism, approval, multi-agent coordination) is added by *editing the graph* — adding nodes and edges — rather than growing one long function.

Nodes that depend on runtime configuration (e.g. a model chosen from a menu) are naturally implemented as **closures** inside a factory function: the factory creates the model and chain once, and the nested node functions capture them.

---

3. State Management and Conditional Routing

3.1 State schema design

Design the state around the data the workflow needs at each stage:

- one field per piece of data (`question`, `cleaned_question`, `category`, `routed_answer`, `final_output`, `messages`, `context`, ...),
- keep raw input alongside derived values for traceability (the original wording stays in `question` while downstream nodes use `cleaned_question`),
- only one writer per key per run unless a reducer is defined,
- prefer simple types (`str`, `list`) for clarity; use richer types (`dict`, message objects) where the framework expects them.

3.2 Reducers

A **reducer** is an annotation on a state field that tells LangGraph how to merge concurrent or successive updates to that field.

- **Without a reducer (default):** overwrite — the returned value replaces the previous one. Correct for scalars like `category` or `final_output`.
- **With a reducer:** the returned value is *combined* with the existing value according to the reducer function.

The canonical example is chat history:

```
class S(TypedDict):
    messages: Annotated[list, add_messages]
```

`add_messages` **appends/merges** instead of replacing: a node returning `{"messages": [new_message]}` extends the list. Without the reducer, the second turn would *replace* the list and the assistant would forget everything from earlier turns.

Another important reducer for parallel workflows is `operator.add`, which concatenates lists — every worker appends its contribution and LangGraph merges them (Section 7.3).

`MessagesState` is a ready-made TypedDict with `messages: Annotated[list, add_messages]` — convenient for chat-style graphs.

Rule of thumb:

Field kind	Merge strategy
Derived scalar written by one node (`category`, `final_output`)	overwrite (no annotation)
Growing collection written by many nodes or turns (`messages`, `findings`)	reducer (`add_messages`, `operator.add`)

3.3 Conditional edges and routing

Conditional edges decide the next node at runtime:

```
builder.add_conditional_edges(
    "classify_query",   # decision happens after this node
    route_query,        # function: state -> path key
    ROUTES,             # path_map: {path key -> destination node}
)
```

- **Routing function** – receives the state and returns a *path key* (not a node name). The path map translates the key into a destination node and doubles as the edge labels in graph drawings.
- **Dynamic routing** – the path is not fixed at build time; it depends on data produced during execution (here: the classification result).

- **Defensive routing** – if the returned key is not in the path map, redirect to a safe default (e.g. `general`). The router must never point at a missing node.
- **Single source of truth** – one routing table dict used by the path map, by the routing function, *and* to validate classifier output, so category names, prompts and nodes cannot drift apart.

3.4 Classification-based (decision) workflows

A common architecture — and the backbone of agents — is **classifier → router → specialist**:

1. **Classifier node** – uses structured LLM output to assign exactly one category from a fixed vocabulary, with reasoning.
2. **Conditional edge** – maps the category to a specialist node.
3. **Specialist nodes** – each runs its own prompt chain (policy, invoice, reimbursement, budget, general, …).
4. **Fan-in** – all specialists converge on a single output-formatting node, then END.

Design details that matter in practice:

- **Node factories** – six specialist nodes often share identical logic (build chain → invoke → return answer); only the system prompt and name differ. A factory function avoids six copies of the same code and sets a meaningful `__name__` so traces show real labels.
- **Model-free fallbacks** – the clarification path for genuinely unclear input can be a fixed template: cheaper, zero tokens, and incapable of hallucinating a policy.
- **Deterministic keyword fallback** – if the structured classification call fails (weak local model, network error, unexpected JSON), a pure-Python keyword classifier still produces a valid category. Order of rules matters (first match wins); very short input maps to "unclear"; everything else falls through to "general". The graph *always* leaves the classifier with a routable value.
- **Cycles** – LangGraph allows edges back to earlier nodes (needed for agent loops). A routing workflow itself can stay acyclic while still using conditional edges.

3.5 Safe destinations

Every routing design should define a **safe destination** for unknown or unsupported inputs — a clarification node, a general handler, or an explicit error path. Unclassifiable input must never reach a dead end or crash.

---

4. Retrieval-Augmented Generation (RAG)

4.1 The need for RAG

A language model knows only what is in its training data (plus what is in the prompt). It does **not** know your company's documents: policy limits, invoice rules, internal procedures. Asked directly, it will either refuse or **hallucinate** plausible-sounding but wrong specifics (invented limits, made-up policy numbers).

Retrieval-Augmented Generation grounds answers in real documents: before answering, the application *retrieves* the relevant passages from a corpus and places them in the prompt as context. The model then answers *from that context* and can cite the source document.

4.2 Architecture: two phases

RAG has two distinct phases:

Index time (build, usually once at startup):

```
.txt / PDF files
   → load into Document objects
   → split into overlapping chunks
   → embed each chunk into a vector
   → store vectors + text in a vector database
```

Query time (per question):

```
question → embed question → similarity search → top-k chunks
         → format into a context string → {context} in the prompt → LLM
```

4.3 Documents and loaders

- A **`Document`** is the standard envelope for text an LLM should see: `page_content` (the text) plus `metadata` (key–value pairs).
- **Loaders** turn files (text, PDF, HTML, SharePoint, …) into Documents. Even when a loader reads whole files manually, the resulting objects are identical — production systems simply swap in purpose-built loaders.
- **Metadata survives splitting.** Each chunk inherits the source file name and a readable policy title, so retrieved chunks — and therefore answers — can cite which document they came from.

4.4 Text splitting (chunking)

Why split at all?

- Embedding models have maximum input lengths.
- One giant vector for a whole policy blurs together unrelated rules; a query about "hotel limits" should match the hotel *section*, not the entire travel policy.

Chunking parameters:

- **`chunk_size`** – maximum characters per chunk. Small = precise matching; large = more context per hit.
- **`chunk_overlap`** – characters repeated between consecutive chunks, so a fact that falls on a boundary still appears (in full) in at least one chunk.
- **Separators** – a *recursive* splitter tries separators in order (`"\n\n"` , `"\n"` , `". "` , `" "` , `""`), so paragraphs and sentences stay intact wherever possible instead of being cut mid-word.

4.5 Embeddings

An **embedding** is a function that maps text to a fixed-length vector of numbers such that *semantically similar texts are close together* in vector space. Two operations matter:

- `embed_documents(list[str])` – vectorise the corpus (index time).
- `embed_query(str)` – vectorise the question (query time).

Both must use the **same embedding model**. Vectors from different models have different dimensions and incompatible geometries — mixing them in one store produces meaningless results (and typically a dimensionality error).

Embedding models are separate from chat models and are also configured via `.env` (a small local model such as a ~274 MB embedder, or a cloud embedding model served under the same API key). Embedding calls happen only when building the index or issuing a search — **retrieval itself involves no LLM call and no chat-token cost.**

4.6 Vector databases

A **vector database** stores vectors alongside their text and answers nearest-neighbour queries — "which stored chunks are closest to this query vector?", typically measured by cosine similarity.

ChromaDB is a lightweight option that integrates directly with LangChain: `Chroma.from_documents(documents, embedding=..., collection_name=...)` embeds and stores everything in one call, and `vectorstore.as_retriever(search_kwargs={"k": 4})` exposes a retriever.

Practical notes:

- **In-memory stores** rebuild the index on every start — simple, always fresh, never stale; production systems add a `persist_directory` to keep the index on disk.
- **One collection = one embedding model.** A robust trick is to derive the collection name *from the embedding model name* (sanitised to legal characters, length-capped). Switching providers then creates a *new* collection instead of silently mixing incompatible vectors.

4.7 Retrievers and similarity search

A **retriever** is simply a Runnable: `str → list[Document]`. It composes with everything else and involves no LLM:

```
chunks = retriever.invoke(question)   # top-k, ranked by similarity
```

- **`k`** – how many chunks to return. Four is often enough context while keeping the prompt small.
- Retrieval is the fastest thing to debug: if the right chunk is not in the results, the problem is the index or the query wording — not the model. A dedicated "show retrieval only" mode (no model call, no tokens) isolates this immediately.

4.8 Formatting context into prompts

Retrieved chunks must be turned into a prompt-friendly string:

- Label each block with its index and source file so the model can name the document it is quoting.
- Apply a **character/token safety cap** so a burst of chunks cannot blow up the prompt (or the free-tier quota): add whole chunks until the cap is reached, then stop.
- Provide an explicit **"no context" message** when nothing was retrieved. This is critical: an empty context invites the model to invent company policy; a clear "no company document matched — answer from general knowledge and say no internal policy was found" prevents that.

Prompt rules for RAG answers typically instruct the model to:

- quote specific limits/amounts from the context and name the source,
- say what is missing when the context does not contain the answer rather than inventing numbers,
- optionally still use its tools for calculations and lookups.

The context itself is usually injected into a `{context}` placeholder of a system message that is **rebuilt every request** — so context appears exactly once while conversation history grows separately (and is not duplicated).

---

5. Tools and Agent Workflows

5.1 What a tool is

RAG retrieves **knowledge** ("what does the policy say?"). Tools perform **operations** — exact arithmetic and record lookups that an LLM must never do itself. Language models are weak at arithmetic (they approximate) and can hallucinate structured records (vendor GSTINs, balances, IDs). A tool is:

> a Python function together with a schema that the model can read, so the model can *request* its execution; Python executes it and returns the exact result.

The model decides **when** and **with what arguments**; Python decides the **result**.

5.2 Defining a tool

The `@tool` decorator converts a normal function into a LangChain tool:

```
@tool
def gst_calculator(base_amount: float, rate_name: str = "standard") -> str:
    """One-line description THE MODEL READS to decide when to call it."""
    ...  # plain Python: exact arithmetic
```

Three ingredients:

1. **Docstring** → becomes the tool description shown to the model. This is the primary signal the model uses to choose a tool — write it as a clear instruction including *when to use it*.
2. **Type hints** → become the JSON schema of the arguments the model must supply.
3. **Return value** → a string (or structured data) that flows back to the model as a tool result.

Reference data used by tools (vendor masters, rate tables, rules) should be loaded once from data files at startup — business data stays out of code.

5.3 Tool catalogue, binding and calls

- **Catalogue** – a list of tools (e.g. expense calculator, GST calculator, budget utilisation, invoice total, vendor lookup). Order defines how the model sees them.
- **`llm.bind_tools(catalogue)`** – appends the tool catalogue (names, descriptions, argument schemas) to every model request. The model then answers either with plain text or with structured **tool calls**.
- **Tool call message** – an `AIMessage` whose `tool_calls` field contains entries like `{name, args}`.
- **Tool result** – after execution, the result is appended to the conversation as a `ToolMessage`, so the model can read it on its next turn.

Not every model supports tool calling. If binding fails, the standard strategy is graceful degradation: keep a tool-free model — retrieval-based answers still work; only the calculators are lost. A status flag can expose whether tool calling is active.

5.4 ToolNode and tools_condition

LangGraph ships prebuilt agent components:

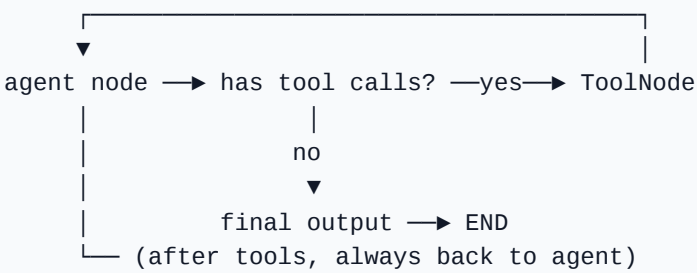
- **`ToolNode(catalogue)`** – a node that reads the `tool_calls` of the last `AIMessage`, executes the matching Python functions, and appends the results. No custom dispatch code is needed.
- **`tools_condition`** – a built-in routing function for conditional edges: returns `"tools"` when the last message has tool calls, `"__end__"` otherwise.

Wiring the loop:

```
builder.add_conditional_edges("agent", tools_condition, {
    "tools": "tools",              # tool call requested → execute tools
    "__end__": "final_output",     # plain answer → format and finish
})
builder.add_edge("tools", "agent") # ALWAYS return to the model
```

Redirecting `"__end__"` to a formatting node (instead of ending the graph immediately) keeps a single presentation path for every finished run.

5.5 The agent execution cycle



1. The agent node builds the request: `[SystemMessage(rules + context)] + conversation history`, invokes the model, and **appends** the reply (text *or* tool calls) to the message list.
2. `tools_condition` inspects the reply.
3. If tools were requested, `ToolNode` executes them and appends results; control returns to the agent so the model sees those results.
4. The model either calls more tools or answers the user.
5. On a plain answer, the run proceeds to final output.

This is an **execution cycle (a graph loop)**. Its superstep count is bounded by a **`recursion_limit`** in the run config (e.g. 40) so a model that requests tools forever cannot spin the process indefinitely.

A typical tool-using question costs: 1 model call to request the tool + 1 tool execution + 1 model call to formulate the final answer — plus zero retrieval LLM calls (embeddings happen at index time).

5.6 ReAct (Reason → Act → Observe)

The agent loop is a concrete instance of the ReAct pattern:

ReAct step	In the graph
Reason	the agent node / system+history prompt → model decides what to do
Act	structured tool calls → ToolNode executes the Python function
Observe	`ToolMessage` result appended to state → fed back to the model
Repeat	until the model produces a final natural-language answer

5.7 RAG vs tools: separate capabilities

	RAG	Tools
Answers	*knowledge* — "what does the policy say?"	*operations* — "calculate / look up this"

	RAG	Tools
Failure mode	nothing retrieved → explicit no-context message	model never calls a tool → plain RAG answer still works
Cost	0 chat calls for the search itself	1 chat call per tool round-trip

Both are needed: a policy-limit question must **cite the document**; a budget percentage must be **computed exactly**. In one agent they complement each other — retrieved context rides in the system message; tools ride in the model's bound tool catalogue.

---

6. Conversation Memory, Checkpointing and Persistence

6.1 Conversational state

In a graph, chat history *is* graph state. The model "remembers" earlier turns simply because those turns live in `state["messages"]` and are resent with every request:

```
request = [SystemMessage(behaviour rules)] + state["messages"]
```

Design points:

- One field holds every `HumanMessage` / `AIMessage` (and `ToolMessage`) of the thread.
- Nodes that produce messages **append** via the `add_messages` reducer; display-only fields like `final_output` keep overwrite semantics (only the latest answer is shown).
- The **system message is rebuilt each turn and never stored in state** — only the conversation is state; instructions are framing. This prevents duplicating rules (or retrieved context) across turns.
- Behaviour prompts for ongoing conversations should tell the model to *use facts from earlier turns* (budgets, amounts, department names) without re-asking, and to show formulas when it computes with user-supplied numbers.

6.2 Threads: isolating conversations

A **thread** is an independent conversation identified by a `thread_id`:

```
config = {"configurable": {"thread_id": "employee-raj"}}
app.invoke({"messages": [("human", question)]}, config=config)
```

- Same id → same history (load before running, save after).
- Different id → completely isolated history.
- One employee, one finance reviewer, one demo user — each gets a unique thread; no code changes needed to separate them.

- Starting a "new conversation" simply means switching to a fresh thread id.
- Listing known threads and resuming an old one are read operations on the checkpointer (SQLite: `SELECT DISTINCT thread_id ...`; memory: keys of the saver's storage).

6.3 Checkpointing

Checkpointing means that after every superstep, LangGraph writes the full state to a *saver* keyed by the `thread_id`. It is enabled at compile time:

```
app = builder.compile(checkpointer=saver)
```

From then on, every `invoke()` / `stream()` **auto-saves and auto-restores** — no extra code inside the nodes.

Why it matters:

- **Process restarts / crashes** – without a saver, state dies with the process; with one, restarting and resuming a thread continues where the user left off ("durable execution" in its simplest form).
- **Multiple users** – each thread's checkpoints are separate rows, so users never see each other's history.
- **Inspection** – `app.get_state(config)` reads the latest snapshot **without running the graph**: no model call, no cost. Unknown threads return an empty snapshot, which callers should handle gracefully.

6.4 Checkpointer types

Checkpointer	Backing store	Persistence	Typical use
InMemorySaver	RAM (dict keyed by thread)	lost when the process exits	demos, tests, zero setup
SqliteSaver	SQLite `.db` file	survives restarts; inspectable with any SQLite tool	real sessions, multi-run continuity

Implementation notes:

- The choice comes from configuration (`.env`), not code; the SQLite import is done lazily so memory-only users never need the extra package.
- The connection object is returned alongside the saver so the CLI can close/flush the file on exit (even on Ctrl+C via `finally`).
- Closing matters: a locked or unflushed database loses checkpoints.
- Provider switches can reuse the same checkpointer — history stays valid while the model changes.

6.5 Context management and growth

History grows every turn, and the full history is resent to the model each time. Consequences:

- The model can reference numbers from many messages ago (this is exactly how a multi-turn budget calculation works: budget stated in turn 1, spend in turn 2, utilisation computed in turn 3).
- Latency and token usage grow with history. Classroom-scale conversations are fine; production systems add windowing or truncation strategies (summarise older turns, keep only the last N messages plus a summary).

Debugging aids that cost nothing: `/history`-style views via `get_state()` (print saved messages instantly), and verifying that messages actually grow (turn counts 2 → 4 → 6 …) to confirm the reducer is active.

Common failure: an assistant that "forgets" the first message almost always means the messages field lacks the `add_messages` reducer and each return overwrites the list.

---

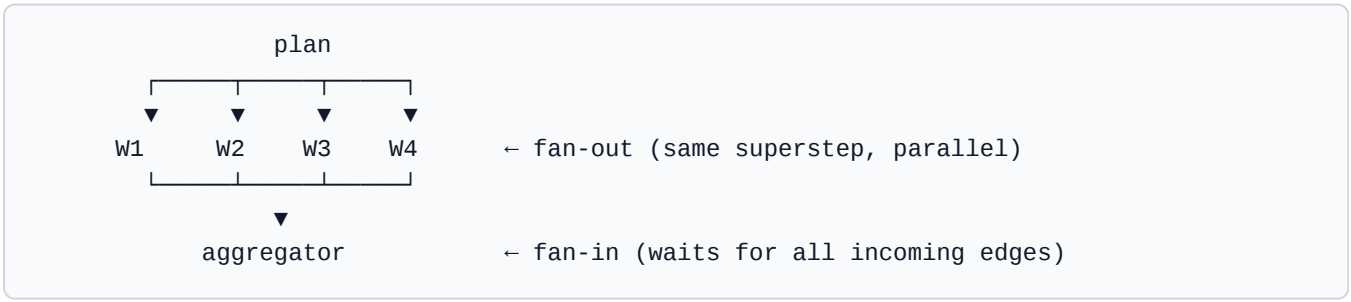
7. Advanced Graph Execution

7.1 Cycles and execution control

- Edges may point backwards, creating **cycles** (the agent loop is the basic example; review → revise → approve loops are another).
- **`recursion_limit`** (set in the run config) bounds how many supersteps a single run may take; exceeding it raises an error instead of hanging. Agent graphs commonly raise the default (e.g. to 40) to allow several tool round-trips.
- Loop-based workflows repeat *until a condition holds*; the condition is evaluated by a conditional edge after each pass.

7.2 Parallel execution: fan-out and fan-in

All destinations of one superstep run **concurrently**:



- **Fan-out** – several `add_edge("X", ...)` calls from one node. No ordering is implied among the destinations; each only depends on state written earlier.
- **Fan-in** – several edges converging on one node. The aggregator runs only after **every** incoming edge has completed.
- **Wall time** – parallel workers finish in approximately the time of the *slowest* worker, not the sum of all workers. (Two 0.3 s workers ≈ 0.31 s total, not 0.6 s.)

- In traces (`stream_mode="updates"`), parallel workers appear as separate updates **with no fixed order between them** — that unordered group is the visible signature of fan-out.

7.3 Collecting parallel results safely

Four concurrent workers writing the *same* state key is an **`InvalidUpdateError`** : the default merge is last-write-wins, and for concurrent nodes there is no meaningful "last". Two standard solutions:

1. **Separate keys per worker** — `budget_report` , `invoice_report` , `reimbursement_report` , `compliance_report` . Each worker owns its key; conflict-free by construction. The aggregator reads all of them.
2. **A reducer on a shared key** — `findings: Annotated[list, operator.add]` . Every worker *appends* its bullets; LangGraph concatenates them in completion order. This is the standard pattern when several nodes contribute to one collection.

Mixing both is common: detailed sections in private keys, short shared bullets in a reduced list. Removing the reducer while multiple workers still write the shared key is precisely the error reducers exist to prevent.

7.4 Dynamic fan-out: the Send API (introduction)

Static fan-out means a fixed number of edges. When the *number* of workers must be decided at runtime (one per planned task), the **Send API** provides dynamic fan-out from a conditional edge:

```
def dispatch(state) -> list[Send]:
    return [Send("worker", {"task": t}) for t in state["tasks"]]
```

Each `Send` carries its own payload to (possibly) the same node — the planner decides how many workers run.

7.5 Node-driven control flow: Command (introduction)

A node that needs to both *write state* and *choose its successor* in one step returns a `Command` :

```
return Command(goto="deep_review", update={"flag": True})
```

This replaces a separate routing function when the decision and the state update naturally belong together. `Command` also underpins interrupt/resume flows (pausing for human approval and later resuming with a supplied value).

7.6 Subgraphs

A **subgraph** is a compiled `StateGraph` used as *one node* of a parent graph.

```
sub_builder = StateGraph(BudgetSubState)
sub_builder.add_node("load", ...); sub_builder.add_node("compute", ...)
sub_builder.add_node("format", ...)
budget_subgraph = sub_builder.compile()

builder.add_node("budget_analysis", budget_subgraph)
```

Properties:

- **Encapsulation** – the parent sees a single node; internal steps stay hidden and can be tested or reused independently.
- **Own state schema** – the subgraph declares its own fields; intermediate keys exist only inside it. When the subgraph finishes, only the keys the parent also declares flow back (e.g. `budget_report`), keeping the parent state clean.
- **Reuse** – the same compiled subgraph can be embedded in any parent workflow.

Example: a budget worker implemented as *load → compute → format* — file read, exact arithmetic, report rendering — registered as one parent node.

7.7 Reliability: retries, timeouts and fallbacks

Multi-step workflows touch files, networks and model APIs; all of them can fail transiently or permanently. Three complementary mechanisms:

Retry with backoff

- Re-run a failed operation up to *n* times with an increasing delay (linear backoff: 0.5 s, 1.0 s, …).
- Transient failures (locked file, brief I/O error, provider hiccup) often succeed on the second try; the delay also gives a busy service time to recover instead of hammering it.
- Log each failed attempt so traces show retries; raise the last error after exhaustion so the caller can decide the fallback.

Timeouts

- Run risky calls (especially LLM invokes that may hang on a slow network or overloaded free-tier endpoint) in a worker thread with a deadline (`future.result(timeout=...)`).
- On timeout, raise `TimeoutError`; the caller falls back — the workflow never stalls indefinitely.
- Practical caveat: an abandoned thread may keep running until it finishes; production code prefers async clients with `wait_for` or per-request executors. A teaching implementation uses the standard library only and cancels the queued future without waiting.

Fallback strategies (graceful degradation)

Every fragile step should have a deterministic backup:

Step	Fallback
LLM planner (structured task plan)	fixed, pre-defined task list; department extracted by regex
LLM aggregator (narrative report)	template report stitched from the deterministic sections
Data-loading workers	"data unavailable" section for that area only
Structured classification	keyword rules
Tool binding	run without tools (RAG still works)
Structured answer	wrap plain text into the same schema

The guiding principle: **one broken dependency must not kill the run.** The graph always completes and always produces output; only quality degrades for the affected part.

7.8 Deterministic computation vs LLM narration

A critical design rule for data-heavy workflows:

> **The LLM never computes; it narrates.**

- Arithmetic (utilisation percentages, totals, variances) and record checks (missing fields, duplicates, PO overruns, rule violations) are done by **pure Python** over real loaded records — exact, testable, side-effect free, and needing no model.
- The LLM's job is to **plan** (decompose a request into typed tasks) and **report** (turn deterministic findings into a readable narrative) — not to invent the numbers.
- Why: an LLM can round 6,50,000 / 10,00,000 incorrectly; a function cannot. Deterministic loaders and analysers also make the parallel workers cheap (0 chat calls) and unit-testable without a model running.

7.9 Structured planning in workflows

A planner node can use structured output to decompose one natural-language request into independent, typed tasks:

- schema example: `TaskPlan { department: str, tasks: [PlannedTask{task_type, instruction}] }` with `task_type ∈ {budget, invoice, reimbursement, compliance}` restricted by `Literal`;
- the validated plan drives which workers run and with what instructions;
- a timeout wraps the call; a fixed fallback plan guarantees work to dispatch even when the model fails or returns junk.

7.10 Debugging graph execution

- `stream_mode="updates"` prints every node update in execution order — the single most useful tool for understanding a graph.
- Expect to see: sequential nodes in order; agent→tools→agent cycles for tool questions; *unordered groups* for parallel workers; the aggregator last in a fan-in.
- **Retrieval-only inspection** isolates RAG quality from prompt/model quality.
- **State snapshots** (`get_state`) show what a thread currently holds without spending tokens.
- Design-level checks: one writer per key (or a reducer), `ROUTES` in sync with registered nodes, `compile()` and `invoke()` both called, recursion limit high enough for the expected loop depth.

---

8. Provider Considerations and Operational Concerns

8.1 Local vs cloud models

Concern	Local (e.g. Ollama)	Cloud (e.g. Gemini)
Cost	free per call; bounded by hardware	metered; free-tier quotas exist
Setup	install server, pull models	API key in `.env`
Privacy	data never leaves the machine	data sent to provider
Capability	smaller models: weaker tool calling / structured output	stronger models, native function calling
Latency	depends on CPU/RAM (small models on CPU are slow)	network + queue dependent
Availability	depends on local service running	depends on quota/uptime

Both are reachable through identical interfaces; the application selects one via configuration and never branches on provider in workflow code.

8.2 Capability differences and defensive coding

Providers differ in:

- **structured output** (function calling vs JSON-schema mode vs neither),
- **tool calling** support (some models reject the tool catalogue),
- **content shape** (string vs list-of-parts — parsers handle both),
- **rate limits** (429 responses → brief waits/retries or switch provider).

Hence the recurring pattern: *try the rich feature; on any failure, degrade to a simpler path that always works.* Every advanced capability in these notes — structured classification, tools, planner, aggregator — has such a fallback.

8.3 Embedding consistency

One embedding model per collection, always. Index and query vectors must come from the same model; deriving the collection name from the model name makes accidental mixing structurally impossible. Never hot-swap an embedding model against an open index — restart and rebuild instead.

8.4 Secrets and configuration hygiene

- Keys and model names live only in `.env`; never in source code, never in terminal transcripts, never in chat logs.
- `.env.example` (placeholders only) is the shareable template.
- Display helpers must show that a key is *configured*, never print the key.
- Runtime databases holding conversation state (synthetic or otherwise) are disposable artifacts — safe to delete; they rebuild on the next run.

8.5 Cost discipline for free-tier usage

Budget the chat calls per interaction:

- simple Q&A: 1 call;
- RAG answer: 1 chat call (+ embedding calls only at index build);
- tool-using answer: 1–3 chat calls (one per tool round-trip) + 0 retrieval calls;
- memory turn: 1 chat call;
- full analysis workflow with LLM planner + LLM narrator: 2 chat calls, with all workers deterministic.

Retrieval searches and `get_state()` inspections are free of chat tokens. Keeping the expensive LLM steps few and pushing work into deterministic code is both cheaper and more reliable.

---

9. Concept Map (summary)

```
LangChain components
  prompts · chat models · parsers · structured output · LCEL runnables · tools
        │
        ▼
LangGraph orchestration
  state (TypedDict) · nodes · edges · START/END · compile/invoke/stream
        │
        ├── sequential pipelines            (input → model → output)
        ├── conditional routing             (classify → route → specialist)
        ├── agent loops                     (agent ⇄ tools, recursion_limit)
        ├── memory + checkpointing          (add_messages, threads, savers)
        └── parallel workflows              (fan-out/fan-in, reducers, subgraphs)
                │
                ▼
Cross-cutting concerns
  RAG grounding · deterministic computation · retries/timeouts/fallbacks
  provider-agnostic design · .env configuration · secret hygiene
```

Development trajectory of an LLM application:

1. **Single-shot chain** — prompt, model, parser; externalised config; structured outputs where fields are needed.
2. **Graph workflow** — explicit state and nodes; same chain inside a node.
3. **Conditional routing** — classification + dynamic edges + safe fallbacks.
4. **Grounding and actions** — RAG for knowledge, tools for operations, agent loop with bounded recursion.
5. **Memory** — message reducers, thread ids, checkpointers, resume.
6. **Scale-out** — parallel fan-out/fan-in, subgraphs, retries, timeouts, layered fallbacks; LLM plans and narrates, code computes.

Each stage adds capability by editing the graph — not by growing an unstructured function — which is the central idea of stateful LLM orchestration.